

```

import numpy as np
from scipy.special import genlaguerre, gammaln
from itertools import product
import scipy.sparse as sp
import scipy.sparse.linalg as sla
import scipy.linalg as la
def ssh_energies(band, omega_q, gc1, gc2, k):
    delta = np.sqrt(gc1**2 + gc2**2 + 2 * gc1 * gc2 * np.cos(k))
    if band == 0:
        return omega_q * np.sqrt(1 + 2 * delta / omega_q)
    else:
        return omega_q * np.sqrt(1 - 2 * delta / omega_q)

def _eye(d):
    return sp.eye(d, format='csr')

def _destroy(d):
    data = np.sqrt(np.arange(1, d, dtype=float))
    return sp.diags(data, 1, shape=(d, d), format='csr')

def _num(d):
    return sp.diags(np.arange(d, dtype=float), 0, format='csr')

def compositions(n, total):
    if n == 1:
        yield [total]
        return
    for first in range(total + 1):
        for rest in compositions(n - 1, total - first):
            yield [first] + rest

def compute_position_exponential_sparse(N, u):
    matrix = np.zeros((N, N), dtype=complex)
    for m in range(N):
        for n in range(N):
            if m >= n:
                matrix[m, n] = (
                    np.exp(-u**2 / 2)
                    * np.exp(0.5 * (gammaln(n + 1) - gammaln(m + 1)))
                    * (1.0j * u) ** (m - n)
                    * genlaguerre(n, m - n)(u**2)
                )
            else:

```

```

        matrix[m, n] = (
            np.exp(-u**2 / 2)
            * np.exp(0.5 * (gammaln(m + 1) - gammaln(n + 1)))
            * (1.0j * u) ** (n - m)
            * genlaguerre(m, n - m)(u**2)
        )
    return sp.csr_matrix(matrix)

class PHOTONICChain:
    """Clase general para la cadena de transmons acoplada al resonador
    Se trabaja en la representación adimensionalizada y se puede elegir si se trabaja con OBC o PBC.
    La idea es diagonalizar la matriz dinámica asociada a la parte capacitiva del hamiltoniano.
    """
    def __init__(self, N, Omega_C, Chi_C, omega_r, gamma, fock_photon, Omega_J, Chi_J, PBC = 0):
        self.N = N
        self.Omega_C = Omega_C
        self.Chi_C = Chi_C
        self.gc1 = gc1 = 0.25 * Omega_C * (1 + Chi_C)
        self.gc2 = gc2 = 0.25 * Omega_C * (1 - Chi_C)
        self.Omega_J = Omega_J
        self.Chi_J = Chi_J
        self.gj1 = gj1 = Omega_J * (1 + Chi_J) # omega_J no tiene un factor 4 como lo tiene el hamiltoniano
        self.gj2 = gj2 = -Omega_J * (1 - Chi_J)
        self.omega_r = omega_r
        self.gamma = gamma
        self.fock_photon = fock_photon

        n = 2 * N
        T = np.zeros((n, n))
        for j in range(n - 1):
            val = gc1 if j % 2 == 0 else gc2
            T[j, j+1] = T[j+1, j] = val
        if PBC and N != 1:
            T[0, n-1] = T[n-1, 0] = gc2

        A = 0.5 * (np.eye(n) + T)
        B = -0.5 * T

        tau_z = np.block([
            [ np.eye(n), np.zeros((n, n)) ],
            [ np.zeros((n, n)), -np.eye(n) ]
        ])

        # Van Hemmen: diagonalize the dynamical matrix  $\hat{I} @ D = \tau_z @ D_{\text{herm}}$ 

```

```

D_herm = np.block([[A, B],
                   [B, A]]) # real symmetric, keep it real

L = np.linalg.cholesky(D_herm) # D_herm = L @ L.T
M_sym = L.T @ tau_z @ L # real symmetric

evals_sym, evects_sym = la.eigh(M_sym) # real evals and evects, sorted

evects_real = la.solve_triangular(L.T, evects_sym) # back to  $x = (L.T)^{-1} y$ 

eigenvals = evals_sym # already real

# Keep positive branch
pos_idx = eigenvals > 1e-15
pos_evals = eigenvals[pos_idx]
pos_evects = evects_real[:, pos_idx]

sort_idx = np.argsort(pos_evals)
pos_evals = pos_evals[sort_idx]
pos_evects = pos_evects[:, sort_idx]

# Symplectic Gram-Schmidt - stays real since inputs are real
def symplectic_gram_schmidt(vecs, tau_z):
    n_cols = vecs.shape[1]
    result = np.zeros_like(vecs, dtype=float) # <-- float, not complex
    for i in range(n_cols):
        v = vecs[:, i].copy()
        for j in range(i):
            e_j = result[:, j]
            proj = e_j @ tau_z @ v # no .conj() needed
            v = v - proj * e_j
        norm = v @ tau_z @ v # should be positive
        result[:, i] = v / np.sqrt(norm)
    return result

U_V = symplectic_gram_schmidt(pos_evects, tau_z)

u_part = U_V[:n, :]
v_part = U_V[n:, :]
J_U_V = np.vstack([v_part, u_part]) # J acts as swap on real vecs

self.P = np.hstack([U_V, J_U_V]) # real pseudo-unitary matrix
self.inverse_P = np.linalg.inv(self.P)
# Transform both Hermitian Hamiltonians into the Bogoliubov basis
# H_C should come out diagonal; H_J generally will not
self.H_C = self.P.T.conj() @ D_herm @ self.P

```

```

TJ = np.zeros((n, n))
for j in range(n - 1):
    val = gj1 if j % 2 == 0 else gj2
    TJ[j, j+1] = TJ[j+1, j] = val
if PBC and N != 1:
    TJ[0, n-1] = TJ[n-1, 0] = gj2

AJ = 0.5 * ((gj1 + gj2) * np.eye(n) - TJ)
BJ = -0.5 * TJ - 0.5 * (gj1 + gj2) * np.eye(n)

H_J_herm = np.block([[AJ, BJ],
                      [BJ, AJ]])
self.H_J = self.P.T.conj() @ H_J_herm @ self.P

def build_HC_fock(self, states, state_index, layer_offsets, n_total):
    n = 2 * self.N
    energies = np.diag(self.H_C)[:n] # <--- Slice the first 'n' physical energies
    rows, cols, data = [], [], []

    def add(row, col, val):
        if abs(val) > 1e-15:
            rows.append(row)
            cols.append(col)
            data.append(val)

    for exc, arr in states.items():
        offset = layer_offsets[exc]
        for s_idx, state in enumerate(arr):
            global_col = offset + s_idx # ket = col
            diag_val = np.dot(energies, state) # <--- Now 6 matches 6
            add(global_col, global_col, diag_val) # diagonal: row == col

    return 2.0 * sp.csr_matrix((data, (rows, cols)), shape=(n_total, n_total)) # no tom

def build_HJ_fock(self, states, state_index, layer_offsets, n_total):
    n = 2 * self.N
    A_J = self.H_J[:n, :n] # b†_i b_j
    B_J = self.H_J[:n, n:] # b†_i b†_j
    C_J = self.H_J[n:, :n] # b_i b_j
    D_J = self.H_J[n:, n:] # b_i b†_j -> treated same as A, no commutation residue

    # CHANGED: dict instead of three lists
    data_dict = {}

    def add(row, col, val):

```

```

if abs(val) > 1e-9:
    if (row, col) in data_dict:
        data_dict[(row, col)] += val    # accumulate
    else:
        data_dict[(row, col)] = val    # first time

# --- Number conserving: A (b†i bj) and D (bi b†j, no delta) ---
# --- Number conserving: A (b†i bj) and D (bi b†j, no delta) ---
# --- Number conserving: AJ (b†i bj) and DJ (bi b†j, no delta) ---
for exc, arr in states.items():
    offset = layer_offsets[exc]
    idx_map = state_index[exc]
    for s_idx, state in enumerate(arr):
        global_col = offset + s_idx

        # AJ[i,j]: apply bj first, then b†i
        for j in range(n):
            if state[j] == 0:
                continue
            factor_j = np.sqrt(state[j])
            for i in range(n):
                val = A_J[i, j]
                if abs(val) < 1e-9:
                    continue
                new_state = state.copy()
                new_state[j] -= 1
                factor_i = np.sqrt(new_state[i] + 1)
                new_state[i] += 1
                key = tuple(new_state)
                if key in idx_map:
                    global_row = offset + idx_map[key]
                    add(global_row, global_col, val * factor_j * factor_i)

        # DJ[i,j] treated as b†j bi (normal ordered, delta dropped)
        for i in range(n):
            if state[i] == 0:
                continue
            factor_i = np.sqrt(state[i])
            for j in range(n):
                val = D_J[i, j]
                if abs(val) < 1e-9:
                    continue
                new_state = state.copy()
                new_state[i] -= 1
                factor_j = np.sqrt(new_state[j] + 1)
                new_state[j] += 1

```

```

        key = tuple(new_state)
        if key in idx_map:
            global_row = offset + idx_map[key]
            add(global_row, global_col, val * factor_i * factor_j)

# --- Pairing: B ( $b_i^\dagger b_j^\dagger$ , +2 excitations) ---
for exc, arr in states.items():
    exc_plus2 = exc + 2
    if exc_plus2 not in states:
        continue
    offset_col = layer_offsets[exc]           # ket lives in exc layer
    offset_row = layer_offsets[exc_plus2]     # bra lives in exc+2 layer
    idx_map_plus = state_index[exc_plus2]
    for s_idx, state in enumerate(arr):
        global_col = offset_col + s_idx
        for j in range(n):
            for i in range(n):
                val = B_J[i, j]
                if abs(val) < 1e-9:
                    continue
                #  $b_i^\dagger b_j^\dagger |state\rangle$ : first  $b_j^\dagger$ , then  $b_i^\dagger$ 
                new_state = state.copy()
                new_state[j] += 1
                factor_j = np.sqrt(new_state[j])
                new_state[i] += 1
                factor_i = np.sqrt(new_state[i])
                key = tuple(new_state)
                if key in idx_map_plus:
                    global_row = offset_row + idx_map_plus[key]
                    add(global_row, global_col, val * factor_j * factor_i)

# --- Pairing: C ( $b_i b_j$ , -2 excitations) ---
for exc, arr in states.items():
    exc_minus2 = exc - 2

    if exc_minus2 not in states:
        continue
    offset_col = layer_offsets[exc]           # ket lives in exc layer
    offset_row = layer_offsets[exc_minus2]     # bra lives in exc-2 layer
    idx_map_minus = state_index[exc_minus2]
    for s_idx, state in enumerate(arr):
        global_col = offset_col + s_idx
        for j in range(n):
            for i in range(n):
                val = C_J[i, j]

```

```

        if abs(val) < 1e-9:
            continue
        # b_i b_j /state>: first b_j, then b_i
        if state[j] == 0:
            continue
        new_state = state.copy()
        new_state[j] -= 1
        factor_j = np.sqrt(state[j])
        # now apply b_i to new_state
        if new_state[i] == 0:
            continue
        factor_i = np.sqrt(new_state[i])
        new_state[i] -= 1
        key = tuple(new_state)
        if key in idx_map_minus:
            global_row = offset_row + idx_map_minus[key]
            add(global_row, global_col, val * factor_j * factor_i)

rows = [k[0] for k in data_dict]
cols = [k[1] for k in data_dict]
data = list(data_dict.values())

return sp.csr_matrix((data, (rows, cols)), shape=(n_total, n_total)) # por como este

def matter_hamiltonians(self, n_excitations, layers):
    n = 2 * self.N
    states = {}
    state_index = {}
    layer_offsets = {}
    global_offset = 0

    for layer_index in range(-layers, layers + 1):
        excitations = n_excitations + 2 * layer_index
        if excitations < 0:
            continue
        arr = np.array([np.array(combo) for combo in compositions(n, excitations)])
        states[excitations] = arr
        state_index[excitations] = {tuple(s): i for i, s in enumerate(arr)}
        layer_offsets[excitations] = global_offset
        global_offset += len(arr)

    n_total = global_offset

    self.fock_states = states
    self.fock_state_index = state_index

```

```

self.fock_offsets      = layer_offsets
self.fock_dim          = n_total

self.H_C_fock = self.build_HC_fock(states, state_index, layer_offsets, n_total)
self.H_J_fock = self.build_HJ_fock(states, state_index, layer_offsets, n_total)

def build_full_hamiltonian(self, linear_coupling=True):
    # --- photon sector ---
    H_ph = self.omega_r * _num(self.fock_photon) #  $\omega_r a^\dagger a$ 

    # --- Interaction operator ---
    if linear_coupling:
        # Linearized coupling:  $(a + a^\dagger)$ 
        sin_op = self.gamma * (_destroy(self.fock_photon) + _destroy(self.fock_photon).adjoint())
    else:
        # Exact non-linear coupling:  $\sin(a + a^\dagger)$ 
        sin_op = -0.5j * (
            compute_position_exponential_sparse(self.fock_photon, self.gamma)
            - compute_position_exponential_sparse(self.fock_photon, -self.gamma)
        )

    eye_ph = _eye(self.fock_photon)
    eye_tr = sp.eye(self.fock_dim, format='csr')

    # --- chain sectors ---
    H0 = self.H_C_fock
    H1 = self.H_J_fock

    # --- combine with kron (photon is the leftmost/slowest index) ---
    self.H = (
        sp.kron(H_ph, eye_tr, format='csr') #  $H_{ph}$   $I_{chain}$ 
        + sp.kron(eye_ph, H0, format='csr') #  $I_{ph}$   $H_C$ 
        - sp.kron(sin_op, H1, format='csr') #  $\sin$   $H_J$ 
    )

def get_low_lying_spectrum(self, k=5):
    """
    Computes the lowest 'k' eigenvalues and eigenvectors of the full Hamiltonian.

    Parameters:
    k (int): Number of low-lying states to compute (ground state + k-1 excited states).
    linear_coupling (bool): Whether to use the linearized cavity coupling.

    Returns:
    evals (ndarray): The 'k' lowest eigenvalues.
    evecs (ndarray): The corresponding 'k' eigenvectors (as columns).
    """

```



```

"""

H = 0.5 * (self.H + self.H.conj().T)

# 3. Solve for the lowest eigenvalues using the Lanczos method
# which='SA' means "Smallest Algebraic" (the most negative/lowest energies)
try:
    evals, evecs = sla.eigsh(H, k=k, which='SA')
except sla.ArpackNoConvergence as e:
    print(f"Warning: ARPACK did not converge. Returning the {len(e.eigenvalues)} fo
    evals, evecs = e.eigenvalues, e.eigenvectors

# ARPACK doesn't always guarantee perfectly sorted output, so we sort them here
idx = np.argsort(evals)
return evals[idx], evecs[:, idx]
# def apply_quadratic_hamiltonian(self, H_bdg, fock_state, states, state_index, layer_o
#
# """
#     Applies a general quadratic BdG Hamiltonian to a single Fock state.
#
#     Parameters
#     -----
#
#     H_bdg      : (2n x 2n) BdG matrix in the normal mode basis
#     fock_state : 1D array of length n, occupation numbers of the input state
#     states     : dict {excitations -> array of Fock states}
#     state_index : dict {excitations -> dict{tuple(state) -> index}}
#     layer_offsets: dict {excitations -> global offset}
#
#     Returns
#     -----
#
#     result : dict {tuple(new_state) -> coefficient}
#             The action of H on |fock_state>, expressed as a sum of Fock states
#
#     """
#     n = 2 * self.N
#     A_J = H_bdg[:n, :n] # b†_i b_j
#     B_J = H_bdg[:n, n:] # b†_i b†_j
#     C_J = H_bdg[n:, :n] # b_i b_j
#     exc = int(np.sum(fock_state))
#     result = {} # tuple(state) -> coefficient
#
#     def add_to_result(state, coeff):
#         key = tuple(state)
#         if key in result:
#             result[key] += coeff
#         else:
#             result[key] = coeff

```

```

# # --- A block: b†i bj ---
# for j in range(n):
#     if fock_state[j] == 0:
#         continue
#     factor_j = np.sqrt(fock_state[j])
#     for i in range(n):
#         val = A_J[i, j]
#         if abs(val) < 1e-9:
#             continue
#         new_state = fock_state.copy()
#         new_state[j] -= 1
#         factor_i = np.sqrt(new_state[i] + 1)
#         new_state[i] += 1
#         add_to_result(new_state, val * factor_j * factor_i)

# # --- B block: b†i b†j (exc -> exc+2) ---
# for j in range(n):
#     for i in range(n):
#         val = B_J[i, j]
#         if abs(val) < 1e-9:
#             continue
#         new_state = fock_state.copy()
#         new_state[j] += 1
#         factor_j = np.sqrt(new_state[j])
#         new_state[i] += 1
#         factor_i = np.sqrt(new_state[i])
#         add_to_result(new_state, val * factor_j * factor_i)

# # --- C block: bi bj (exc -> exc-2) ---
# for j in range(n):
#     if fock_state[j] == 0:
#         continue
#     for i in range(n):
#         val = C_J[i, j]
#         if abs(val) < 1e-9:
#             continue
#         if fock_state[j] == 0:
#             continue
#         new_state = fock_state.copy()
#         new_state[j] -= 1
#         factor_j = np.sqrt(fock_state[j])
#         if i == j and new_state[i] == 0:
#             continue
#         if new_state[i] == 0 and i != j:
#             continue
#         factor_i = np.sqrt(new_state[i] if i != j else fock_state[i] - 1)

```

```

#             new_state[i] -= 1
#             if new_state[i] < 0:
#                 continue
#             add_to_result(new_state, val * factor_j * factor_i)

#     # --- filter near-zero coefficients ---
#     result = {k: v for k, v in result.items() if abs(v) > 1e-9}

#     return result

def build_squeezed_vacuum(self, truncation):
    """
    Constructs the Bogoliubov vacuum in the old Fock basis via the expansion:

     $|0\rangle_{\text{new}} = N \exp(-0.5 \sum_i b_{i-}^\dagger M_{ij} b_{j-}^\dagger) |0\rangle_{\text{old}}$ 

    where  $M = U_{\text{tilde}}^{-1} V_{\text{tilde}}$  (Blaizot convention)

    Parameters
    -----
    truncation : int
        Number of pair-creation terms to include beyond the vacuum.
        truncation=0 -> only  $|0\rangle$ 
        truncation=1 ->  $|0\rangle$  + two-particle terms
        truncation=k -> up to 2k-particle terms

    Returns
    -----
    state_coeff_dict : dict {tuple(fock_state) -> coefficient}
    """
    n = 2 * self.N
    U_tilde = self.inverse_P[:, n, :n]
    V_tilde = self.inverse_P[:, n, n:]
    M = np.linalg.solve(U_tilde, V_tilde) #  $U_{\text{tilde}}^{-1} V_{\text{tilde}}$ , more stable than

    vacuum = tuple(np.zeros(n, dtype=int))
    state_coeff_dict = {}
    state_coeff_dict[vacuum] = 1.0

    # Each term in the Taylor expansion of  $\exp(-0.5 \sum_i b_{i-}^\dagger M_{ij} b_{j-}^\dagger)$  is:
    #  $(1/k!) * (-0.5)^k * (\sum_i b_{i-}^\dagger M_{ij} b_{j-}^\dagger)^k |0\rangle$ 
    # We build this iteratively: at each order k we act  $(\sum_i b_{i-}^\dagger M_{ij} b_{j-}^\dagger)$  on all
    # states from order k-1 and accumulate with the prefactor  $1/k * (-0.5)$ 

    current_terms = {vacuum: 1.0} # states generated at the previous order

```

```

for k in range(1, truncation + 1):
    next_terms = {}

    def add(d, key, val):
        if key in d:
            d[key] += val
        else:
            d[key] = val

    # act  $(-0.5 * b_i^\dagger M_{ij} b_j^\dagger)$  on each state in current_terms
    # prefactor for this order:  $1/k$  from the Taylor series (the  $(-0.5)$  is in  $M$  appl
    for state_tuple, coeff in current_terms.items():
        state = np.array(state_tuple, dtype=int)
        for j in range(n):
            for i in range(n):
                val = M[i, j]
                if abs(val) < 1e-9:
                    continue
                new_state = state.copy()
                new_state[j] += 1
                factor_j = np.sqrt(new_state[j])
                new_state[i] += 1
                factor_i = np.sqrt(new_state[i])
                new_key = tuple(new_state)
                # prefactor:  $(-0.5)$  from the exponent,  $1/k$  from Taylor expansion
                add(next_terms, new_key, coeff * (-0.5) * val * factor_j * factor_i

    # accumulate into full dict
    for key, val in next_terms.items():
        if abs(val) > 1e-9:
            if key in state_coeff_dict:
                state_coeff_dict[key] += val
            else:
                state_coeff_dict[key] = val

    current_terms = next_terms

    # --- normalize ---
    norm = np.sqrt(sum(abs(v)**2 for v in state_coeff_dict.values()))
    state_coeff_dict = {k: v / norm for k, v in state_coeff_dict.items()}

    self.vacuum = state_coeff_dict

def apply_eta_dag(self, i, state_coeff_dict):
    """
    Applies  $\eta_i^\dagger = U_{ij} b_j^\dagger + V_{ij} b_j$  to all states in state_coeff_dict.

```

Parameters

i : mode index for η_i
state_coeff_dict : dict {tuple(fock_state) -> coefficient}

Returns

result : dict {tuple(fock_state) -> coefficient}
"""

```
n      = 2 * self.N
U      = self.P[:n, :n]  # particle block
V      = self.P[:n, n:]  # hole block
```

```
result = {}
```

```
def add(d, key, val):
    if abs(val) < 1e-14:
        return
    if key in d:
        d[key] += val
    else:
        d[key] = val
```

```
for state_tuple, coeff in state_coeff_dict.items():
    state = np.array(state_tuple, dtype=int)
```

```
# U_ij b†_j part: creates one excitation in mode j
for j in range(n):
    val = U[i, j]
    if abs(val) < 1e-14:
        continue
    new_state = state.copy()
    new_state[j] += 1
    factor = np.sqrt(new_state[j])
    add(result, tuple(new_state), coeff * val * factor)
```

```
# V_ij b_j part: destroys one excitation in mode j
for j in range(n):
    val = V[i, j]
    if abs(val) < 1e-14:
        continue
    if state[j] == 0:
        continue
    new_state = state.copy()
    factor = np.sqrt(state[j])
```

```

        new_state[j] -= 1
        add(result, tuple(new_state), coeff * val * factor)

    return result

def new_to_old_fock(self, new_fock_state):
    """
    Translates a Fock state in the new (Bogoliubov) basis to the old basis.

     $|n\rangle_{\text{new}} = \prod_i [(\eta_i^\dagger)^{r_i} / r_i!] |0\rangle_{\text{new}}$ 

    Parameters
    -----
    new_fock_state : array-like of length n, occupation numbers in the new basis

    Returns
    -----
    state_coeff_dict : dict {tuple(fock_state) -> coefficient}
                      normalized superposition in the old basis
    """
    n = 2 * self.N
    occupations = np.array(new_fock_state, dtype=int)

    # start from the squeezed vacuum expressed in the old basis
    state_coeff_dict = dict(self.vacuum) # copy so we don't mutate

    # apply  $\eta_i^\dagger$   $r_i$  times for each mode i, dividing by  $r_i!$  cumulatively
    for i in range(n):
        r_i = occupations[i]
        if r_i == 0:
            continue
        for k in range(1, r_i + 1):
            state_coeff_dict = self.apply_eta_dag(i, state_coeff_dict)
            # divide by k at each step -> total factor 1/r_i! after r_i steps
            state_coeff_dict = {s: v / k for s, v in state_coeff_dict.items()}

    # normalize
    norm = np.sqrt(sum(abs(v)**2 for v in state_coeff_dict.values()))
    if norm > 1e-14:
        state_coeff_dict = {k: v / norm for k, v in state_coeff_dict.items()}

    return state_coeff_dict

def quanta_distribution(self, fock_vector, state_coeff_dict=None):
    """

```

Computes the quanta probability distribution over old bosonic modes.

Given a state $|\Psi\rangle = \sum_s c_s |s\rangle$, computes:

*$n_i = \sum_s |c_s|^2 * s_i$ (expected occupation per mode)
then normalizes so that $\sum_i n_i = 1$.*

Parameters

*-----
fock_vector : dict {tuple(fock_state) -> coefficient}
OR 1D array in the global Fock basis ordering*

Returns

*-----
distribution : 1D array of length n, normalized quanta distribution
"""*

*n = 2 * self.N*

```

if isinstance(fock_vector, dict):
    state_coeff_dict = fock_vector
else:
    # convert global vector to dict using stored basis
    state_coeff_dict = {}
    for exc, arr in self.fock_states.items():
        offset = self.fock_offsets[exc]
        for s_idx, state in enumerate(arr):
            coeff = fock_vector[offset + s_idx]
            if abs(coeff) > 1e-15:
                state_coeff_dict[tuple(state)] = state_coeff_dict.get(tuple(state),
distribution = np.zeros(n)
for state_tuple, coeff in state_coeff_dict.items():
    distribution += (np.abs(coeff)**2) * np.array(state_tuple)

total = distribution.sum()
if total > 1e-15:
    distribution /= total

return distribution

```

```

def quanta_distribution_subspace(self, fock_vector, n_excitations, cumulative=False):
    """

```

*Computes the quanta probability distribution restricted to a given
excitation subspace (or cumulatively up to that subspace).*

Parameters

```

-----
fock_vector      : 1D array in the global Fock basis ordering
n_excitations    : int, target excitation number
cumulative       : bool, if True include all subspaces up to n_excitations

Returns
-----
distribution : 1D array of length n, normalized quanta distribution
"""
n = 2 * self.N
distribution = np.zeros(n)

# determine which excitation layers to include
if cumulative:
    target_layers = [exc for exc in self.fock_states if exc <= n_excitations]
else:
    target_layers = [n_excitations] if n_excitations in self.fock_states else []

for exc in target_layers:
    arr = self.fock_states[exc]
    offset = self.fock_offsets[exc]
    for s_idx, state in enumerate(arr):
        coeff = fock_vector[offset + s_idx]
        distribution += (np.abs(coeff)**2) * np.array(state, dtype=float)

total = distribution.sum()
if total > 1e-15:
    distribution /= total

return distribution

def quanta_ipr(self, fock_vector):
    """
    Computes the quanta-weighted Inverse Participation Ratio (IPR):


$$IPR = \sum_i \langle n_i \rangle^2 / (\sum_i \langle n_i \rangle)^2$$


    where  $\langle n_i \rangle = \sum_s |c_s|^2 * s_i$  is the expected occupation of mode i.

    IPR -> 1/n_modes : fully delocalized
    IPR -> 1          : fully localized on one mode

Parameters
-----
fock_vector : 1D array in the global Fock basis ordering

```



```

Returns
-----

ipr : float, the IPR value
xi : float, effective localization length (= 1/IPR in units of modes)
distribution : 1D array, the unnormalized  $\langle n_i \rangle$  used for the IPR
"""
n = 2 * self.N
distribution = np.zeros(n)

for exc, arr in self.fock_states.items():
    offset = self.fock_offsets[exc]
    for s_idx, state in enumerate(arr):
        coeff = fock_vector[offset + s_idx]
        distribution += (np.abs(coeff)**2) * np.array(state, dtype=float)

total = distribution.sum()
if total < 1e-15:
    return 0.0, np.inf, distribution

ipr = np.sum(distribution**2) / (total**2)
xi = 1.0 / ipr # effective number of modes occupied

return ipr, xi, distribution

def decompose_tensor_state(self, full_vector):
    """
    Decomposes a state vector living in  $H_{\text{photon}}$   $H_{\text{matter}}$  into a
    dictionary keyed by photon Fock index, where each value is itself
    a dict {tuple(bogoliubov_fock_state) -> complex coefficient}.

    The global index ordering from build_full_hamiltonian is:
        idx = alpha * fock_dim + m
    (photon is the slow/left index in sp.kron).

    Parameters
    -----
    full_vector : 1D array of length fock_photon * fock_dim

    Returns
    -----
    decomposition : dict { alpha (int) -> dict { tuple(matter_state) -> complex } }
        For each photon Fock number alpha, the corresponding slice of the
        wavefunction expressed in the Bogoliubov matter Fock basis.
    """
    d_ph = self.fock_photon

```

```

d_mat = self.fock_dim

assert len(full_vector) == d_ph * d_mat, (
    f"Vector length {len(full_vector)} != fock_photon*fock_dim = {d_ph*d_mat}"
)

# Reshape: rows = photon index alpha, cols = matter index m
psi = full_vector.reshape(d_ph, d_mat)

decomposition = {}
for alpha in range(d_ph):
    matter_slice = psi[alpha, :] # length fock_dim
    matter_dict = {}
    for exc, arr in self.fock_states.items():
        offset = self.fock_offsets[exc]
        for s_idx, state in enumerate(arr):
            coeff = matter_slice[offset + s_idx]
            if abs(coeff) > 1e-15:
                matter_dict[tuple(state)] = matter_dict.get(tuple(state), 0.0) + coeff
    if matter_dict:
        decomposition[alpha] = matter_dict

return decomposition

def decompose_to_old_fock(self, full_vector):
    """
    Takes a state in  $H_{\text{photon}}$   $H_{\text{matter}}$  (Bogoliubov basis for matter)
    and expresses it in  $H_{\text{photon}}$   $H_{\text{matter}}$  (old bosonic basis),
    by applying new_to_old_fock to each photon sector independently.

    Requires self.vacuum to be set (call build_squeezed_vacuum first).

    Parameters
    -----
    full_vector : 1D array of length fock_photon * fock_dim

    Returns
    -----
    old_decomposition : dict { alpha (int) -> dict { tuple(old_fock_state) -> complex } }
        For each photon Fock number alpha, the matter part expressed in
        the old (pre-Bogoliubov) bosonic Fock basis, as returned by
        new_to_old_fock. Coefficients are NOT renormalized across alpha
        sectors so that the global norm is preserved.
    """
    bogoliubov_decomp = self.decompose_tensor_state(full_vector)

```

```

old_decomposition = {}
for alpha, matter_dict in bogoliubov_decomp.items():
    # matter_dict is a superposition in the Bogoliubov basis:
    # |matter> =  $\sum_s c_s |s>_{\text{new}}$ 
    # new_to_old_fock handles single Fock states, so we must
    # superpose their images weighted by the coefficients.
    old_matter = {}
    for new_state_tuple, coeff in matter_dict.items():
        # new_to_old_fock expects a Fock occupation array in the new basis
        old_part = self.new_to_old_fock(np.array(new_state_tuple, dtype=int))
        # old_part is internally normalized - we must weight by coeff
        # and accumulate into old_matter
        norm_old = np.sqrt(sum(abs(v)**2 for v in old_part.values()))
        for old_state, old_coeff in old_part.items():
            weighted = coeff * old_coeff / (norm_old if norm_old > 1e-15 else 1.0)
            old_matter[old_state] = old_matter.get(old_state, 0.0) + weighted

    old_decomposition[alpha] = old_matter

return old_decomposition

def partial_trace(self, full_vector, which_to_keep='matter'):
    """
    Computes the reduced density matrix by tracing out one subsystem.

    The full Hilbert space is  $H_{\text{photon}} \otimes H_{\text{matter}}$ , with global index
     $\text{id}x = \alpha * \text{fock\_dim} + m$ 
    (photon is the slow/left index from sp.kron).

    Parameters
    -----
    full_vector : 1D array of length fock_photon * fock_dim
    which_to_keep : 'matter' or 'photon'

    Returns
    -----
    rho : 2D dense array of shape (fock_dim, fock_dim) if which_to_keep == 'matter'
        (fock_photon, fock_photon) if which_to_keep == 'photon'
    """
    d_ph = self.fock_photon
    d_mat = self.fock_dim

    psi = full_vector.reshape(d_ph, d_mat) # (alpha, m)

```

```

if which_to_keep == 'matter':
    #  $\rho_{\text{matter}_{\{m,m'\}}} = \sum_{\alpha} \psi[\alpha, m] * \psi[\alpha, m'].conj()$ 
    return  $\psi.conj().T @ \psi$  # (fock_dim, fock_dim)
elif which_to_keep == 'photon':
    #  $\rho_{\text{photon}_{\{\alpha, \alpha'\}}} = \sum_m \psi[\alpha, m] * \psi[\alpha', m].conj()$ 
    return  $\psi @ \psi.conj().T$  # (fock_photon, fock_photon)
else:
    raise ValueError(f"which_to_keep must be 'matter' or 'photon', got '{which_to_k")

```